

1.3 Listen

Listen haben den gleichen Grundaufbau wie **Queues** – Es wird nur eine Referenz auf den Kopf gehalten und die einzelnen Knoten sind als _____ miteinander **verkettet**.

Die Liste kann jedoch mehr - so kann an beliebiger Stelle eingefügt, entfernt oder auslesen werden.

Hierzu erweitern wir außerdem die Funktionen der Klasse **Node**

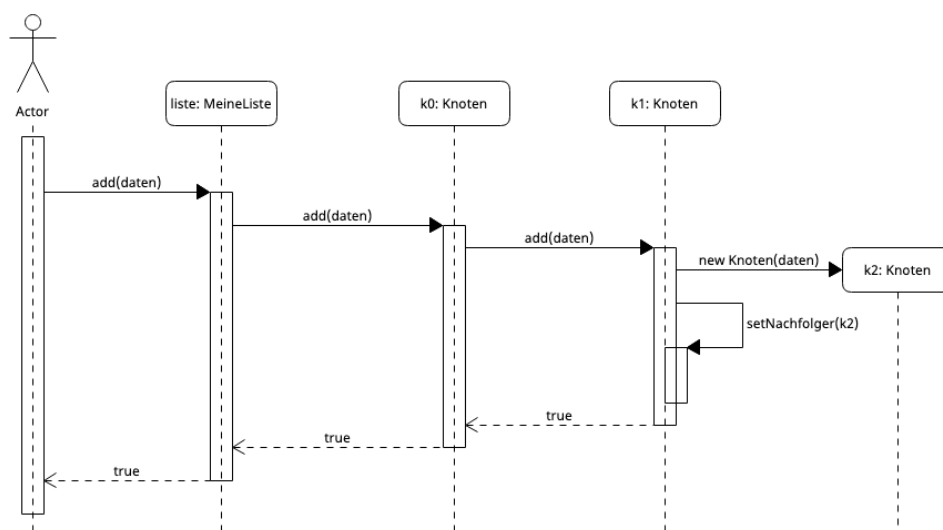
Einfügen am Datenelement durch Durchreichen

```
boolean add(DatenElement daten) {  
    if(nachfolger == null) {  
        nachfolger = new Knoten(daten);  
        return true;  
    } else {  
        // Selbstaufruf --> Rekursion  
        return nachfolger.add(daten);  
    }  
}
```

Aufruf der gleichen Methode innerhalb der Methode führt zu einer Verschachtelung -> **Rekursiver Aufruf**. Die Methode gibt einfach dem nächsten Knoten den gleichen Auftrag weiter.

Von _____ spricht man, wenn innerhalb einer Methode die Methode selbst erneut aufgerufen wird. Dies läuft theoretisch _____, deshalb muss eine _____ eingeführt werden, welche dann greift, wenn die Methode erfolgreich umgesetzt werden kann (hier _____) oder die Bedingung nicht mehr erfüllt werden kann (z.B. festgelegte Länge überschritten).

Die **Verschachtelung** ist auch sehr gut im _____ zu erkennen:



Aufgabe 1) Benennen Sie die Elemente im Sequenzdiagramm mit Fachbegriffen.

Aufgabe 2)

- Duplizieren Sie ihren Workspace für die Taxiqueue und verändern Sie die Namen mittels Rechtsklick „Rename Symbol“ und passen Sie das Projekt so an, dass es eine Liste umsetzt (nur Referenz auf Anfang).
- Übertragen Sie den Programmcode für die Methode `add(DatenElement daten)` in Ihr Projekt und adaptieren Sie entsprechend die Methode in der Klasse `MeineListe`.
- Ergänzen Sie die Methoden `länge()` und `entfernen(DatenElement daten)` in `MeineListe` und `Knoten` mit der entsprechenden Funktionalität. Übertragen Sie die wichtigsten Codefragmente/-ideen unten.

A full-page sheet of white graph paper featuring a uniform grid of thin black horizontal and vertical lines, creating small squares across the entire surface.

Aufgabe 3: Zeichnen Sie das Sequenzdiagramm zu Aufruf von `länge()` auf einem Objekt `liste` der Klasse `MeineListe`, welche drei Knoten umfasst.

[illegible]

1.3 Listen - Lösung

Listen haben den gleichen Grundaufbau wie **Queues** – Es wird nur eine Referenz auf den Kopf gehalten und die einzelnen Knoten sind als **rekursive Datenstruktur** miteinander **verkettet**. Die Liste kann jedoch mehr - so kann an beliebiger Stelle eingefügt, entfernt oder auslesen werden.

Hierzu erweitern wir außerdem die Funktionen der Klasse **Node**

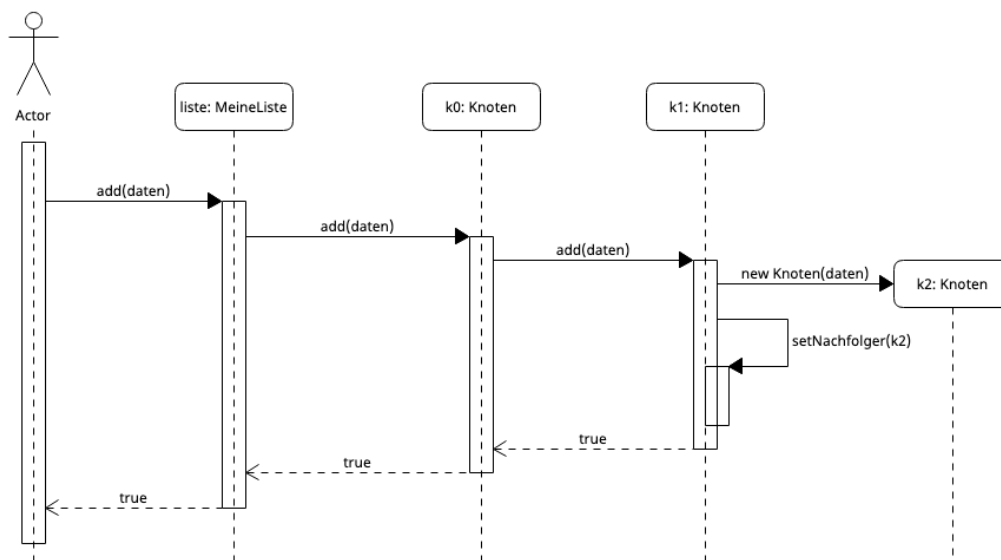
Einfügen am Datenelement durch Durchreichen

```
public boolean add(DataElement data) {  
    if(successor == null) {  
        successor = new Node(data);  
        return true;  
    } else {  
        return successor.add(data); // Selbstaufzuruf --> Rekursion  
    }  
}
```

Aufruf der gleichen Methode innerhalb der Methode führt zu einer Verschachtelung -> **Rekursiver Aufruf**. Die Methode gibt einfach dem nächsten Knoten den gleichen Auftrag weiter.

Von **Rekursion** spricht man, wenn innerhalb einer Methode die Methode selbst erneut aufgerufen wird. Dies läuft theoretisch **unendlich**, deshalb muss eine **Abbruchbedingung** eingeführt werden, welche dann greift, wenn die Methode erfolgreich umgesetzt werden kann (hier **am passenden bzw. letzten Knoten**) oder die Bedingung nicht mehr erfüllt werden kann (z.B. festgelegte Länge überschritten).

Die Verschachtelung ist auch sehr gut im **UML-Sequenzdiagramm** zu erkennen:



Lebenslinien, Akteure, Objekte; Aktivitätsbalken; synchroner Aufruf / Nachricht; Rückgaben / Antwort;

Aufgabe 1: Benennen Sie die Elemente im Sequenzdiagramm mit Fachbegriffen.

Aufgabe 2)

- Duplizieren Sie ihren Workspace für die Taxiqueue und verändern Sie die Namen mittels Rechtsklick „Rename Symbol“ und passen Sie das Projekt so an, dass es eine Liste umsetzt (nur Referenz auf Anfang).
- Übertragen Sie den Programmcode für die Methode `add(DatenElement daten)` in Ihr Projekt und adaptieren Sie entsprechend die Methode in der Klasse `MeineListe`.
- Ergänzen Sie die Methoden `länge()` und `entfernen(DatenElement daten)` in `MeineListe` und `Knoten` mit der entsprechenden Funktionalität. Übertrage den Code unten.

class `MeineListe`

```
int Länge() {  
    if(anfang == null) {  
        //leere Liste  
        return 0;  
    } else {  
        //Rekursion  
        return anfang.Länge();  
    }  
}
```

```
boolean entfernen(DatenElement daten) {  
    if(anfang != null) {  
        // Liste nicht leer  
        if(anfang.getData() == daten) {  
            // betreffender Knoten ist Anfang  
            anfang = anfang.getNachfolger();  
            return true;  
        } else {  
            //Rekursion  
            return anfang.entfernen(daten) != null;  
        }  
    } else {  
        //Leere Liste  
        return false;  
    }  
}
```

class `Knoten`:

```
int Länge() {  
    if(nachfolger == null) {  
        // Ende der Liste  
        return 1;  
    } else {  
        // Rekursion  
        return nachfolger.Länge() + 1;  
    }  
}
```

```
Knoten entfernen(DatenElement daten) {  
    if(this.daten == daten) {  
        // ich habe die Daten  
        return nachfolger;  
    } else {  
        if(nachfolger != null) {  
            // Nachfolger vorhanden, rekursion  
            nachfolger = nachfolger.entfernen(daten);  
            return this;  
        } else {  
            // kein Nachfolger -> Abbruch  
            return null;  
        }  
    }  
}
```

Aufgabe 3: Zeichnen Sie das Sequenzdiagramm zu Aufruf von `länge()` auf einem Objekt `liste` der Klasse `MeineListe`, welche drei Knoten umfasst.

